BioInf 202627 Project 1

Genetic Algorithm for Multiple Sequence Alignment

Introduction

This document describes the first project of Bioinformatics 202627 @NOVAFCT

Key information

- **Deadline:** 29/10/2026 23:59
- **Groups:** two students
- **Submission:** code and report (max. 6 pages) in a single zip file, through moodle
- **Project discussion:** individual and mandatory, after the deadline

Project Information

The goal of this project assignment is to implement a Genetic Algorithm to optimize Multiple Sequence Alignments (MSA). It is assumed you already understand basic biological sequence processing, Pairwise Sequence Alignment algorithms, and general MSA algorithms.

The genetic MSA algorithm to be implemented for this project is based on the following paper:

Notredame, Cédric, and Desmond G. Higgins. "SAGA: sequence alignment by genetic algorithm." Nucleic acids research 24.8 (1996): 1515-1524. https://doi.org/10.1093/nar/24.8.1515

Overall recommendations

Read this document and then read the paper previously mentioned (focus on the algorithm description). Make sure you understand the problem at hand.

You can use any programming language you want, as long as the report says exactly how the code should be run (Question 11 of the Report section).

You do not need to implement the algorithm presented in the paper exactly as it is: some aspects can be simplified or adapted if necessary.

Start by implementing the functions enumerated in the Structure section of this document. Test your functions with simple test cases, such as the small sequences found in the slides and in Rosalind problems. Then run on the provided fasta input file and explore the parameters. Note that not all sequences in this file have the same length.

Your algorithm should work for both DNA and protein sequences.

To make the work more reproducible, set the seed parameter of the random functions you use.

Project resources

You can use the code available on moodle, as well as the code you developed for the lab exercises. An example FASTA file will also be provided (cytochromes.fa), as well as the BLOSUM62 matrix (blosum62.mat).

Concepts and objectives

This project requires the knowledge of concepts discussed during the classes, such as Multiple Sequence Alignment (MSA) and scoring functions.

In the context of the genetic algorithm, a population is a set of individuals. In this case, an individual is a specific way to align the input sequences, i.e., an MSA. All individuals of a population have the same set of sequences, but aligned differently. Note that the sequences should never change - the only thing that changes is where the gaps are placed.

Over each generation (iteration of the outer loop), the population is changed - this is the selection phase, where some individuals are kept from one generation to another, while new ones are created using the mutation and crossover operators.

In the context of this project, a mutation is a function that transforms one individual into another individual by changing where the gaps are placed. Crossover is a function that generates a new individual by combining two individuals (also known as breeding). If crossover generates two new individuals, only the best one should be kept.

Structure

Every operator (crossover and mutation) must produce a valid MSA. Think about what makes an MSA valid and how you can make sure your implementation never breaks it.

Your code should implement at least the following:

Population initialization

Your algorithm should take as input a FASTA file with an arbitrary number of sequences (DNA or protein) of arbitrary length. Then it should produce random MSAs on those sequences.

The reference paper above creates random MSA by adding a random number of gaps at the beginning of each sequence. You can follow this approach to create a function to initialise an MSA population. Note that this may leave some columns containing only gaps, which should be removed as that would not happen in a valid MSA.

Scoring

The reference paper uses the sum of pairs score. You can do the same using the BLOSUM62 matrix and adapting the code used to score MSA with progressive algorithms (Lecture 8, Lab3). It should also be possible to score DNA alignments by providing match and mismatch values. Affine gap penalties are optional.

Selection

At each generation you need to select the parents of the next generation. The reference paper uses stochastic remainder selection (better individuals have a better chance of being chosen) but you can use different selection schemes if you prefer.

Make sure the best individuals are never lost between generations: a fraction of the best MSAs of each generation should be copied unchanged to the next one. The reference paper keeps the best half of the population.

Crossover

The characteristics of the parents must be combined in the offspring. You can use the crossover operator developed in Lab4.

Mutation

To add more diversity to the population you need at least one mutation operator. The operators described in the paper are not trivial and some include local search optimization. Instead **you must select one of the suggested mutation operators from the moodle poll and attempt to implement it**. You are encouraged to implement other mutation operators, **but the one you chose is mandatory.**

Termination

Your algorithm should end at a maximum number of generations or when there is no improvement in the best MSA after a number of generations. It should then return the best MSA found and its score.

Your code should output to the console the score of the best MSA and any other statistics you find relevant, such as the number of generations. Furthermore, the actual alignment should be written to a text file.

Report

You must submit a short report about your project with succinct answers to the following questions, one by one:

1. How do you initialize an MSA population? Explain the logic.
2. How did you score MSA? Explain the function and parameters.
3. How do you select the MSAs of the next generation?

4. Which MSA mutation operators did you implement? (mention the one selected by your group and additional ones if applicable; briefly explain how they work with an example)
5. Which data structures did you use to represent an MSA and the population, and why?
6. How are the crossover and mutation operators used in your algorithm?
7. What are the stopping criteria of your algorithm?
8. Which dataset(s) did you use to test your algorithm?
9. Did your algorithm improve the MSA scores (calculated with your scoring function) over randomly generated MSAs? Design experiments that support your answer and justify your choices, and discuss the contribution of your mutation operators.
10. How was Generative AI used in this project?
11. Instructions to run your code and default values of all parameters

The report should have at most 6 pages and can be written in Portuguese or English. Note that your report should not have anything else, except your group number and names. It does not need a cover, introduction, or conclusion. Only the answers to those questions will be evaluated.

Evaluation Criteria

In line with the Structure and Report sections, the following aspects of your project will be evaluated:

- initialization
- scoring
- selection
- crossover
- mutation(s)
- termination
- general code structure
- report (correctness of answers)
- scope (how much was explored)

Project discussion

After submitting your project, we will schedule individual project discussions. Note that since these discussions are individual, every element of the group must understand every aspect of the project, even if the work was split.

The objective of this discussion is to validate that you accomplished the learning objectives associated with this project. You will be asked to run your code, analyze results, and perform small modifications.

This discussion is mandatory and will also be part of the evaluation assessment. The time slots will be shared after the submission deadline is over. If none of the slots work for you, a different time can be scheduled.

Project Submission

You must submit your code and report through the moodle submission form, inside a single zip file.

Late submission

Submitting your project within 24 hours after the respective deadline will reduce your grade by 25%. Submitting between 24 and 72 hours after the deadline will reduce your grade by 50%. After 72 hours, no submissions will be allowed.

Usage of AI tools

The usage of AI tools (ChatGPT, Copilot, Gemini, etc) is allowed on components that are not critical to the main objectives of the project. This means that the main functions mentioned in Structure section should be implemented without using genAI tools. The answers given in the report should also be written by yourself, while genAI can be used for minor language improvements. If you are not familiar with your project, your evaluation will be negatively impacted.

Any use of these tools should be mentioned in the report, as requested in the Report section. Failure to do so might negatively impact your evaluation.

Plagiarism

Collaboration between different groups is not permitted. In case of plagiarism all groups involved will fail the course. You can use all the code provided in the slides and on the moodle page.

The same applies to code from outside sources (e.g. online repositories, existing implementations of genetic algorithms or MSA tools): it must not be used for the main functions mentioned in the Structure section. Any other external code must be clearly identified and its source cited, both in the code and in the report.